

Trabajo Practico Integrador - Primera entrega parcial

Food Store - Base de Datos II

Estudiante: Santiago Barretto

Repositorio: DB_Santiago_Barretto

Motor validado: PostgreSQL 18.4

Resumen. Esta entrega integra integridad, transacciones y concurrencia; optimizacion de consultas; e indices, vistas y objetos programables. Todas las afirmaciones de ejecucion referencian salidas versionadas del motor.

1. Cobertura de la consigna

Punto	Implementacion verificable
1. Modelo ER	Diagrama, claves, cardinalidad y participacion: tpi/docs/01_modelo_er.md.
2. ER a relacional	FKs 1:N y entidad asociativa detalle_pedido para la N:M.
3. Normalizacion	Dependencias funcionales y justificacion 3FN/BCNF.
4. DDL	schema.sql y migracion idempotente tpi/01_migracion_modelo_final.sql.
5. DML y consultas	JOIN, agregacion, subconsultas, GROUP BY/HAVING y RANK en TP3/TP4.
6. Vistas y PL/pgSQL	Vistas, materializada, 2 funciones y 2 procedimientos.
7. Reglas	CHECK, UNIQUE, trigger de vigencia y trigger de auditoria.
8. Transacciones	Atomicidad, COMMIT/ROLLBACK, aislamiento y bloqueo en dos sesiones.
9. Borrado logico	deleted_at, indices parciales y vistas de entidades vigentes.

La trazabilidad completa de archivos se encuentra en tpi/docs/04_checklist_tpi.md.

2. Modelo y reglas de negocio

El modelo contiene cliente, categoria, producto, pedido y detalle_pedido. La PK compuesta de detalle_pedido preserva cantidad y precio historico. Las FK usan ON DELETE RESTRICT; cliente y producto incorporan deleted_at para preservar el historial sin devolver bajas en las nuevas vistas.

El esquema final agrega nombres no vacios, precios positivos, categoria unica, email unico entre clientes vigentes e indices parciales para las consultas filtradas por deleted_at. El rol sin login food_store_reporter solo recibe SELECT sobre las vistas de reporte.

3. Objetos programables y prueba atomica

sp_tpi_registrar_pedido bloquea el producto, valida cliente y stock, descuenta unidades y crea pedido mas detalle. Ante stock insuficiente lanza excepcion sin dejar un pedido parcial. sp_tpi_baja_logica_cliente marca la baja. Los triggers impiden productos no vigentes en detalles y auditan cambios de producto.

La prueba reversible finalizo con ROLLBACK y confirmo 2 procedimientos, 2 triggers y 2 vistas. Tambien comprobó los tres CHECK, la auditoria y el reuso de email posterior a una baja logica.

4. Transacciones y concurrencia observadas

Caso	Resultado real en PostgreSQL 18.4
Lectura no repetible	READ COMMITTED: 10.00 -> 20.00. REPEATABLE READ: 10.00 -> 10.00.
Lectura fantasma	READ COMMITTED: 2 -> 3 filas activas. REPEATABLE READ: 2 -> 2.
Bloqueo de fila	B inicio 20:30:49.634868-03; adquirio 20:30:52.401280-03, al liberar A.

La ejecucion completa se conserva en tpi/evidencia/20260923_214500/06_concurrencia.txt. El ejecutor abre dos sesiones psql y limpia las filas de laboratorio al finalizar.

5. Optimizacion, vistas y resultados

Trabajo	Resultado documentado
TP3 y TP4	Planes antes/despues, equivalencias y decisiones de indices se preservan en sus carpetas de evidencia.
TP5 - indices	Detalles por producto: 27.891 ms -> 0.152 ms; autocompletado: 8.988 ms -> 1.446 ms; apellido: 3.518 ms -> 0.176 ms.
TP5 - escritura	INSERT reversible de 600 detalles: 8.299 ms sin los indices TP5 y 15.327 ms con ellos; costo reconocido y justificado.
TP5 - materializada	Reporte de facturacion: 550.464 ms -> 0.067 ms; refresco concurrente documentado.

En la verificacion TPI se versionaron ademas los planes crudos, las ocho equivalencias EXCEPT = 0, la medicion de escritura y el refresh en `tp5/evidencia/20260923_tpi/`. Los tiempos pueden variar por cache; los planes y controles conservan la evidencia de la decision.

6. Uso responsable de IA

Codex/OpenAI se utilizo para estructurar scripts, documentacion y controles reproducibles. OpenCode reviso las propuestas de indices de TP5. Cada propuesta se acepto o descarto luego de verificarla contra el esquema y salidas reales de PostgreSQL; los registros estan versionados en los DUIA.

Reproduccion: seguir `tpi/README.md` sobre una copia aislada. Los scripts y las salidas de evidencia permiten inspeccionar cada requisito sin depender de valores inventados.